SQL

SkillSphere Database Queries

Database Tables Creation

```sql
-- Create database
CREATE DATABASE skillsphere;

-- Connect to the database
\c skillsphere

-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Create ENUM types
CREATE TYPE user_role AS ENUM ('student', 'faculty', 'staff', 'admin');
CREATE TYPE proficiency_level AS ENUM ('beginner', 'intermediate',
'advanced');
CREATE TYPE delivery_mode AS ENUM ('online', 'in-person', 'both');
CREATE TYPE urgency_level AS ENUM ('low', 'medium', 'high');
CREATE TYPE exchange_status AS ENUM ('pending', 'accepted', 'completed',
'canceled');

-- Create Users table
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE CHECK (email LIKE '%@aui.ma'),
    password_hash VARCHAR(255) NOT NULL,
    role user_role NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

-- Create User Profiles table
CREATE TABLE user_profiles (
    profile_id SERIAL PRIMARY KEY,
    user_id INTEGER UNIQUE NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    full_name VARCHAR(100) NOT NULL,
    department_major VARCHAR(100),
```

```
    contact_preference VARCHAR(50),
    bio TEXT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Create Categories table
CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT
);

-- Create Skills table
CREATE TABLE skills (
    skill_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    category_id INTEGER NOT NULL REFERENCES categories(category_id) ON DELETE
RESTRICT,
    description TEXT
);

-- Create User Skills table (junction table)
CREATE TABLE user_skills (
    user_skill_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    skill_id INTEGER NOT NULL REFERENCES skills(skill_id) ON DELETE CASCADE,
    proficiency_level proficiency_level NOT NULL,
    notes TEXT,
    UNIQUE(user_id, skill_id)
);

-- Create Skill Offerings table
CREATE TABLE skill_offerings (
    offering_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    skill_id INTEGER NOT NULL REFERENCES skills(skill_id) ON DELETE RESTRICT,
    title VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    mode delivery_mode NOT NULL,
    availability VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

```sql
-- Create Skill Requests table
CREATE TABLE skill_requests (
    request_id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    skill_id INTEGER NOT NULL REFERENCES skills(skill_id) ON DELETE RESTRICT,
    title VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    urgency urgency_level NOT NULL,
    preferred_timeframe VARCHAR(255) NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
);

-- Create Exchanges table
CREATE TABLE exchanges (
    exchange_id SERIAL PRIMARY KEY,
    provider_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
    requester_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE
RESTRICT,
    offering_id INTEGER NOT NULL REFERENCES skill_offerings(offering_id) ON
DELETE RESTRICT,
    request_id INTEGER REFERENCES skill_requests(request_id) ON DELETE SET
NULL,
    status exchange_status NOT NULL DEFAULT 'pending',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (provider_id != requester_id)
);

-- Indexes for performance
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_user_profiles_user_id ON user_profiles(user_id);
CREATE INDEX idx_skills_category_id ON skills(category_id);
CREATE INDEX idx_user_skills_user_id ON user_skills(user_id);
CREATE INDEX idx_user_skills_skill_id ON user_skills(skill_id);
CREATE INDEX idx_skill_offerings_user_id ON skill_offerings(user_id);
CREATE INDEX idx_skill_offerings_skill_id ON skill_offerings(skill_id);
CREATE INDEX idx_skill_requests_user_id ON skill_requests(user_id);
CREATE INDEX idx_skill_requests_skill_id ON skill_requests(skill_id);
CREATE INDEX idx_exchanges_provider_id ON exchanges(provider_id);
CREATE INDEX idx_exchanges_requester_id ON exchanges(requester_id);
CREATE INDEX idx_exchanges_offering_id ON exchanges(offering_id);
CREATE INDEX idx_exchanges_request_id ON exchanges(request_id);
```

Data Population

```
-- Populate Categories
INSERT INTO categories (name, description) VALUES
('Academic', 'Academic skills related to coursework, research, and university
subjects'),
('Technical', 'Technical skills including programming, design, and
engineering'),
('Arts', 'Creative and artistic skills such as music, painting, and
photography'),
('Practical', 'Everyday practical skills like cooking, budgeting, and
organization');

-- Populate Skills
INSERT INTO skills (name, category_id, description) VALUES
-- Academic skills
('Calculus Tutoring', 1, 'Help with calculus concepts, problem-solving, and
exam preparation'),
('Research Paper Writing', 1, 'Assistance with academic research papers,
citations, and formatting'),
('Statistical Analysis', 1, 'Guidance with statistical methods, data analysis,
and interpretation'),
('Arabic Language', 1, 'Teaching conversational and written Arabic for
beginners or intermediate learners'),
('Physics Help', 1, 'Assistance with physics concepts, homework, and lab
reports'),

-- Technical skills
('Web Development', 2, 'Frontend and backend web development using various
frameworks'),
('Mobile App Development', 2, 'Creating mobile applications for iOS and
Android platforms'),
('Graphic Design', 2, 'Design of visual content for print and digital media'),
('Database Management', 2, 'Setting up, optimizing, and managing relational
databases'),
('Python Programming', 2, 'Python coding for data science, automation, and web
development'),

-- Arts skills
('Digital Photography', 3, 'Techniques for taking and editing digital
photographs'),
('Guitar Lessons', 3, 'Teaching acoustic or electric guitar for various music
styles'),
('Creative Writing', 3, 'Fiction, poetry, and creative non-fiction writing
techniques'),
```

```
('Painting', 3, 'Introduction to painting techniques with acrylics, oils, or
watercolors'),
('Vocal Training', 3, 'Voice lessons for singing or public speaking
improvement'),

-- Practical skills
('Cooking', 4, 'Teaching basic to advanced cooking techniques and recipes'),
('Financial Planning', 4, 'Guidance on personal finance management, budgeting,
and saving'),
('Public Speaking', 4, 'Techniques to improve presentation skills and reduce
speaking anxiety'),
('Fitness Training', 4, 'Personalized workout routines and fitness guidance'),
('Time Management', 4, 'Strategies for improving productivity and
organization');

-- Populate Users (with hashed passwords - in real system, these would be
properly hashed)
INSERT INTO users (email, password_hash, role) VALUES
('john.smith@aui.ma',
'$2a$12$vkLOmtyMJ0WE52ZGnK4.Leaz1mVNZJk/X8GWU.HgzZ1K1bAQCwpey', 'student'),
('sarah.johnson@aui.ma',
'$2a$12$qEJu4Eie0ACGBI57HySahu5YvGBCKKF3QyLVMBRQ3rEbHXgk4AGTe', 'student'),
('mohammed.ali@aui.ma',
'$2a$12$9YmKoKbAMNLhJz6MQwsK4.AQkOgTYgmqRQ5XC9wG4RPvYmJUizUYS', 'faculty'),
('fatima.zahra@aui.ma',
'$2a$12$hZKWAMvsWl.1hlnCP58kw.v2NB7RLIpkSQVIMfb3Nk9dBvbLNKUU2', 'staff'),
('admin@aui.ma',
'$2a$12$g1lGKe5gU7A0EOu7WsKEFuw.tAFwHbqPMo8QUpnKzNQbHzAc.L6Y2', 'admin');

-- Populate User Profiles
INSERT INTO user_profiles (user_id, full_name, department_major,
contact_preference, bio) VALUES
(1, 'John Smith', 'Computer Science', 'Email', 'CS student passionate about AI
and machine learning.'),
(2, 'Sarah Johnson', 'Business Administration', 'WhatsApp', 'Business student
with interest in entrepreneurship.'),
(3, 'Mohammed Ali', 'Mathematics Department', 'Email', 'Professor specializing
in advanced calculus and differential equations.'),
(4, 'Fatima Zahra', 'Library Services', 'Phone', 'Library staff with
background in information sciences.'),
(5, 'System Administrator', 'IT Department', 'Email', 'Platform administrator
responsible for system management.');

-- Populate User Skills
INSERT INTO user_skills (user_id, skill_id, proficiency_level, notes) VALUES
(1, 5, 'intermediate', 'Completed physics courses with excellent grades'),
```

```
(1, 10, 'advanced', 'Working on data science projects for 3 years'),
(2, 17, 'intermediate', 'Experienced in public presentations and debate
club'),
(2, 18, 'beginner', 'Recently started learning about personal finance'),
(3, 1, 'advanced', 'Teaching calculus for over 10 years'),
(3, 3, 'advanced', 'Expert in statistical methods and software'),
(4, 14, 'intermediate', 'Hobby painter for 5 years'),
(4, 16, 'advanced', 'Passionate home cook specializing in Moroccan cuisine');

-- Populate Skill Offerings
INSERT INTO skill_offerings (user_id, skill_id, title, description, mode,
availability) VALUES
(1, 10, 'Python Programming Tutoring', 'Offering one-on-one sessions to learn
Python fundamentals and data science applications', 'both', 'Weekdays after
5pm and weekends'),
(2, 17, 'Public Speaking Workshop', 'Group sessions focused on improving
presentation skills and overcoming stage fright', 'in-person', 'Saturday
mornings, 9-11am'),
(3, 1, 'Calculus Crash Course', 'Intensive review sessions for students
struggling with calculus concepts', 'both', 'Tuesday and Thursday evenings'),
(3, 3, 'Statistical Analysis Help', 'Assistance with statistical methods and
software for research projects', 'online', 'By appointment'),
(4, 16, 'Moroccan Cooking Classes', 'Learn to prepare traditional Moroccan
dishes with authentic techniques', 'in-person', 'Sunday afternoons');

-- Populate Skill Requests
INSERT INTO skill_requests (user_id, skill_id, title, description, urgency,
preferred_timeframe) VALUES
(1, 4, 'Arabic Language Practice Partner', 'Looking for someone to practice
conversational Arabic with regularly', 'medium', 'Flexible, preferably
weekends'),
(2, 9, 'Database Help Needed', 'Need assistance setting up a simple database
for a class project', 'high', 'Within the next week'),
(2, 14, 'Painting Lessons for Beginner', 'Interested in learning basic
painting techniques and color theory', 'low', 'Any evening in the coming
months'),
(4, 6, 'Website Creation Assistance', 'Would like help creating a simple
personal website', 'medium', 'Anytime in the next two weeks');

-- Populate Exchanges
INSERT INTO exchanges (provider_id, requester_id, offering_id, request_id,
status) VALUES
(3, 1, 3, NULL, 'accepted'),
(1, 2, 1, 2, 'pending'),
(4, 2, 5, 3, 'completed');
```

Retrieval Queries

```sql
-- 1. Find all active skill offerings with provider information
SELECT o.offering_id, o.title, o.description, o.mode, o.availability,
       s.name AS skill_name, c.name AS category,
       u.email AS provider_email, p.full_name AS provider_name
FROM skill_offerings o
JOIN skills s ON o.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
JOIN users u ON o.user_id = u.user_id
JOIN user_profiles p ON u.user_id = p.user_id
WHERE o.is_active = TRUE
ORDER BY o.created_at DESC;

-- 2. Find all active skill requests with requester information
SELECT r.request_id, r.title, r.description, r.urgency, r.preferred_timeframe,
       s.name AS skill_name, c.name AS category,
       u.email AS requester_email, p.full_name AS requester_name
FROM skill_requests r
JOIN skills s ON r.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
JOIN users u ON r.user_id = u.user_id
JOIN user_profiles p ON u.user_id = p.user_id
WHERE r.is_active = TRUE
ORDER BY
    CASE
        WHEN r.urgency = 'high' THEN 1
        WHEN r.urgency = 'medium' THEN 2
        ELSE 3
    END,
    r.created_at DESC;

-- 3. Search for offerings by skill name, category, or keyword in description
SELECT o.offering_id, o.title, s.name AS skill_name, c.name AS category,
       p.full_name AS provider_name
FROM skill_offerings o
JOIN skills s ON o.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
JOIN users u ON o.user_id = u.user_id
JOIN user_profiles p ON u.user_id = p.user_id
WHERE o.is_active = TRUE AND (
    LOWER(s.name) LIKE LOWER('%python%') OR
    LOWER(c.name) = LOWER('Technical') OR
    LOWER(o.description) LIKE LOWER('%programming%')
)
```

```sql
ORDER BY o.created_at DESC;

-- 4. Find all skills offered by a specific user
SELECT s.name AS skill_name, o.title, o.mode, o.availability
FROM skill_offerings o
JOIN skills s ON o.skill_id = s.skill_id
JOIN users u ON o.user_id = u.user_id
WHERE u.email = 'mohammed.ali@aui.ma' AND o.is_active = TRUE
ORDER BY o.created_at DESC;

-- 5. Find all exchanges for a user (both as provider and requester)
SELECT
    e.exchange_id,
    e.status,
    e.created_at,
    CASE
        WHEN e.provider_id = 2 THEN 'provider'
        ELSE 'requester'
    END AS user_role,
    prov_profile.full_name AS provider_name,
    req_profile.full_name AS requester_name,
    o.title AS offering_title,
    s.name AS skill_name
FROM exchanges e
JOIN users prov ON e.provider_id = prov.user_id
JOIN user_profiles prov_profile ON prov.user_id = prov_profile.user_id
JOIN users req ON e.requester_id = req.user_id
JOIN user_profiles req_profile ON req.user_id = req_profile.user_id
JOIN skill_offerings o ON e.offering_id = o.offering_id
JOIN skills s ON o.skill_id = s.skill_id
WHERE e.provider_id = 2 OR e.requester_id = 2
ORDER BY e.created_at DESC;
```

Aggregate and Grouping Queries

```sql
-- 1. Count offerings and requests by category
SELECT
    c.name AS category,
    COUNT(DISTINCT o.offering_id) AS offering_count,
    COUNT(DISTINCT r.request_id) AS request_count
FROM categories c
LEFT JOIN skills s ON c.category_id = s.category_id
LEFT JOIN skill_offerings o ON s.skill_id = o.skill_id AND o.is_active = TRUE
LEFT JOIN skill_requests r ON s.skill_id = r.skill_id AND r.is_active = TRUE
GROUP BY c.category_id, c.name
```

```sql
ORDER BY c.name;

-- 2. Most popular skills based on offerings and requests
SELECT
    s.name AS skill_name,
    c.name AS category,
    COUNT(DISTINCT o.offering_id) AS offering_count,
    COUNT(DISTINCT r.request_id) AS request_count,
    COUNT(DISTINCT o.offering_id) + COUNT(DISTINCT r.request_id) AS
total_count
FROM skills s
JOIN categories c ON s.category_id = c.category_id
LEFT JOIN skill_offerings o ON s.skill_id = o.skill_id AND o.is_active = TRUE
LEFT JOIN skill_requests r ON s.skill_id = r.skill_id AND r.is_active = TRUE
GROUP BY s.skill_id, s.name, c.name
HAVING COUNT(DISTINCT o.offering_id) + COUNT(DISTINCT r.request_id) > 0
ORDER BY total_count DESC;

-- 3. User activity statistics
SELECT
    p.full_name,
    u.email,
    u.role,
    COUNT(DISTINCT o.offering_id) AS offerings_created,
    COUNT(DISTINCT r.request_id) AS requests_created,
    COUNT(DISTINCT e_prov.exchange_id) AS provided_exchanges,
    COUNT(DISTINCT e_req.exchange_id) AS requested_exchanges
FROM users u
JOIN user_profiles p ON u.user_id = p.user_id
LEFT JOIN skill_offerings o ON u.user_id = o.user_id
LEFT JOIN skill_requests r ON u.user_id = r.user_id
LEFT JOIN exchanges e_prov ON u.user_id = e_prov.provider_id
LEFT JOIN exchanges e_req ON u.user_id = e_req.requester_id
WHERE u.is_active = TRUE
GROUP BY u.user_id, p.full_name, u.email, u.role
ORDER BY (COUNT(DISTINCT o.offering_id) + COUNT(DISTINCT r.request_id) +
          COUNT(DISTINCT e_prov.exchange_id) + COUNT(DISTINCT
e_req.exchange_id)) DESC;

-- 4. Exchange success rates by skill category
SELECT
    c.name AS category,
    COUNT(e.exchange_id) AS total_exchanges,
    COUNT(CASE WHEN e.status = 'completed' THEN 1 END) AS completed_exchanges,
    ROUND(COUNT(CASE WHEN e.status = 'completed' THEN 1 END)::numeric /
          NULLIF(COUNT(e.exchange_id), 0)::numeric * 100, 2) AS
```

```
completion_rate
FROM exchanges e
JOIN skill_offerings o ON e.offering_id = o.offering_id
JOIN skills s ON o.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
GROUP BY c.category_id, c.name
ORDER BY completion_rate DESC;

-- 5. Monthly growth in platform usage
SELECT
    TO_CHAR(date_trunc('month', created_at), 'YYYY-MM') AS month,
    COUNT(DISTINCT u.user_id) AS new_users,
    COUNT(DISTINCT o.offering_id) AS new_offerings,
    COUNT(DISTINCT r.request_id) AS new_requests,
    COUNT(DISTINCT e.exchange_id) AS new_exchanges
FROM (
    SELECT generate_series(
        date_trunc('month', MIN(created_at)),
        date_trunc('month', MAX(created_at)),
        interval '1 month'
    ) AS created_at
    FROM (
        SELECT created_at FROM users
        UNION ALL
        SELECT created_at FROM skill_offerings
        UNION ALL
        SELECT created_at FROM skill_requests
        UNION ALL
        SELECT created_at FROM exchanges
    ) AS all_dates
) dates
LEFT JOIN users u ON date_trunc('month', u.created_at) = dates.created_at
LEFT JOIN skill_offerings o ON date_trunc('month', o.created_at) =
dates.created_at
LEFT JOIN skill_requests r ON date_trunc('month', r.created_at) =
dates.created_at
LEFT JOIN exchanges e ON date_trunc('month', e.created_at) = dates.created_at
GROUP BY date_trunc('month', dates.created_at)
ORDER BY month;
```

Data Manipulation Operations

```
-- 1. Insert a new user with profile
BEGIN;
```

```sql
INSERT INTO users (email, password_hash, role)
VALUES ('new.student@aui.ma', '$2a$12$hashed_password_here', 'student')
RETURNING user_id INTO @new_user_id;

INSERT INTO user_profiles (user_id, full_name, department_major,
contact_preference, bio)
VALUES (@new_user_id, 'New Student', 'Business Analytics', 'WhatsApp',
'Graduate student interested in data analysis and visualization.');

COMMIT;

-- 2. Update a user profile
UPDATE user_profiles
SET
    department_major = 'Computer Science',
    bio = 'CS student with focus on cybersecurity and networking.',
    updated_at = CURRENT_TIMESTAMP
WHERE user_id = 1;

-- 3. Create a new skill offering
INSERT INTO skill_offerings (
    user_id, skill_id, title, description, mode, availability
)
VALUES (
    1, 10, 'Python for Data Analysis',
    'Learn Python for data manipulation, analysis, and visualization using
pandas, numpy, and matplotlib.',
    'online', 'Monday and Wednesday evenings, 7-9pm'
);

-- 4. Update an offering's details
UPDATE skill_offerings
SET
    title = 'Advanced Python for Data Science',
    description = 'Deep dive into Python libraries for machine learning and
data science: scikit-learn, TensorFlow, and more.',
    availability = 'Tuesday and Thursday evenings, 6-8pm',
    updated_at = CURRENT_TIMESTAMP
WHERE offering_id = 1;

-- 5. Deactivate (soft delete) an offering
UPDATE skill_offerings
SET
    is_active = FALSE,
    updated_at = CURRENT_TIMESTAMP
WHERE offering_id = 5;
```

```sql
-- 6. Create a skill request
INSERT INTO skill_requests (
    user_id, skill_id, title, description, urgency, preferred_timeframe
)
VALUES (
    1, 8, 'Logo Design Needed',
    'Looking for help designing a simple logo for a student club project',
    'medium', 'Within the next two weeks'
);

-- 7. Update a request's urgency and timeframe
UPDATE skill_requests
SET
    urgency = 'high',
    preferred_timeframe = 'As soon as possible',
    updated_at = CURRENT_TIMESTAMP
WHERE request_id = 2;

-- 8. Create a new exchange
INSERT INTO exchanges (
    provider_id, requester_id, offering_id, request_id, status
)
VALUES (
    1, 4, 1, 4, 'pending'
);

-- 9. Update an exchange status
UPDATE exchanges
SET
    status = 'accepted',
    updated_at = CURRENT_TIMESTAMP
WHERE exchange_id = 2;

-- 10. Hard delete a user (with cascading effect)
DELETE FROM users
WHERE user_id = 5 AND role != 'admin';
```

Views

```sql
-- 1. Active Skills Marketplace View
CREATE OR REPLACE VIEW vw_active_skills_marketplace AS
SELECT
    'offering' AS listing_type,
    o.offering_id AS listing_id,
```

```
    o.title,
    o.description,
    o.created_at,
    s.skill_id,
    s.name AS skill_name,
    c.category_id,
    c.name AS category_name,
    u.user_id AS provider_id,
    p.full_name AS provider_name,
    o.mode,
    o.availability AS timeframe,
    NULL::text AS urgency
FROM skill_offerings o
JOIN skills s ON o.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
JOIN users u ON o.user_id = u.user_id
JOIN user_profiles p ON u.user_id = p.user_id
WHERE o.is_active = TRUE

UNION ALL

SELECT
    'request' AS listing_type,
    r.request_id AS listing_id,
    r.title,
    r.description,
    r.created_at,
    s.skill_id,
    s.name AS skill_name,
    c.category_id,
    c.name AS category_name,
    u.user_id AS requester_id,
    p.full_name AS requester_name,
    NULL::delivery_mode AS mode,
    r.preferred_timeframe AS timeframe,
    r.urgency::text
FROM skill_requests r
JOIN skills s ON r.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
JOIN users u ON r.user_id = u.user_id
JOIN user_profiles p ON u.user_id = p.user_id
WHERE r.is_active = TRUE;

-- 2. User Skills Profile View
CREATE OR REPLACE VIEW vw_user_skills_profile AS
SELECT
```

```sql
    u.user_id,
    p.full_name,
    u.email,
    u.role,
    s.skill_id,
    s.name AS skill_name,
    us.proficiency_level,
    c.name AS category_name,
    (SELECT COUNT(*) FROM skill_offerings so
     WHERE so.user_id = u.user_id AND so.skill_id = s.skill_id AND
so.is_active = TRUE) AS active_offerings
FROM users u
JOIN user_profiles p ON u.user_id = p.user_id
JOIN user_skills us ON u.user_id = us.user_id
JOIN skills s ON us.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
WHERE u.is_active = TRUE;

-- 3. Exchange Activity View
CREATE OR REPLACE VIEW vw_exchange_activity AS
SELECT
    e.exchange_id,
    e.status,
    e.created_at,
    e.updated_at,
    prov.user_id AS provider_id,
    prov_p.full_name AS provider_name,
    req.user_id AS requester_id,
    req_p.full_name AS requester_name,
    o.offering_id,
    o.title AS offering_title,
    s.name AS skill_name,
    c.name AS category_name,
    r.request_id,
    CASE WHEN r.request_id IS NOT NULL THEN r.title ELSE NULL END AS
request_title
FROM exchanges e
JOIN users prov ON e.provider_id = prov.user_id
JOIN user_profiles prov_p ON prov.user_id = prov_p.user_id
JOIN users req ON e.requester_id = req.user_id
JOIN user_profiles req_p ON req.user_id = req_p.user_id
JOIN skill_offerings o ON e.offering_id = o.offering_id
JOIN skills s ON o.skill_id = s.skill_id
JOIN categories c ON s.category_id = c.category_id
LEFT JOIN skill_requests r ON e.request_id = r.request_id;
```

```sql
-- 4. Admin Dashboard View
CREATE OR REPLACE VIEW vw_admin_dashboard AS
SELECT
    (SELECT COUNT(*) FROM users WHERE is_active = TRUE) AS active_users,
    (SELECT COUNT(*) FROM skill_offerings WHERE is_active = TRUE) AS
active_offerings,
    (SELECT COUNT(*) FROM skill_requests WHERE is_active = TRUE) AS
active_requests,
    (SELECT COUNT(*) FROM exchanges WHERE status = 'pending') AS
pending_exchanges,
    (SELECT COUNT(*) FROM exchanges WHERE status = 'accepted') AS
accepted_exchanges,
    (SELECT COUNT(*) FROM exchanges WHERE status = 'completed') AS
completed_exchanges,
    (SELECT COUNT(*) FROM exchanges WHERE status = 'canceled') AS
canceled_exchanges,
    (SELECT COUNT(*) FROM skills) AS total_skills,
    (SELECT COUNT(*) FROM categories) AS total_categories;
```

Stored Procedures, Functions, and Triggers

```sql
-- 1. Function to calculate user activity score
CREATE OR REPLACE FUNCTION calculate_user_activity_score(user_id_param
INTEGER)
RETURNS INTEGER AS $$
DECLARE
    total_score INTEGER := 0;
    offerings_count INTEGER;
    requests_count INTEGER;
    provider_exchanges_count INTEGER;
    requester_exchanges_count INTEGER;
    completed_exchanges_count INTEGER;
BEGIN
    -- Count offerings (2 points each)
    SELECT COUNT(*) INTO offerings_count
    FROM skill_offerings
    WHERE user_id = user_id_param;

    total_score := total_score + (offerings_count * 2);

    -- Count requests (1 point each)
    SELECT COUNT(*) INTO requests_count
    FROM skill_requests
    WHERE user_id = user_id_param;
```

```
    total_score := total_score + requests_count;

    -- Count exchanges as provider (3 points each)
    SELECT COUNT(*) INTO provider_exchanges_count
    FROM exchanges
    WHERE provider_id = user_id_param;

    total_score := total_score + (provider_exchanges_count * 3);

    -- Count exchanges as requester (2 points each)
    SELECT COUNT(*) INTO requester_exchanges_count
    FROM exchanges
    WHERE requester_id = user_id_param;

    total_score := total_score + (requester_exchanges_count * 2);

    -- Bonus for completed exchanges (5 points each)
    SELECT COUNT(*) INTO completed_exchanges_count
    FROM exchanges
    WHERE (provider_id = user_id_param OR requester_id = user_id_param)
    AND status = 'completed';

    total_score := total_score + (completed_exchanges_count * 5);

    RETURN total_score;
END;
$$ LANGUAGE plpgsql;

-- 2. Function to find matching offerings for a request
CREATE OR REPLACE FUNCTION find_matching_offerings(request_id_param INTEGER)
RETURNS TABLE(
    offering_id INTEGER,
    title VARCHAR,
    provider_name VARCHAR,
    skill_name VARCHAR,
    match_score INTEGER
) AS $$
DECLARE
    req_skill_id INTEGER;
    req_user_id INTEGER;
BEGIN
    -- Get the request's skill ID and user ID
    SELECT skill_id, user_id INTO req_skill_id, req_user_id
    FROM skill_requests
    WHERE request_id = request_id_param;
```

```
    RETURN QUERY
    SELECT
        o.offering_id,
        o.title,
        p.full_name AS provider_name,
        s.name AS skill_name,
        -- Calculate match score (exact skill match = 10 points, same category
= 5 points)
        CASE
            WHEN o.skill_id = req_skill_id THEN 10
            ELSE 5
        END AS match_score
    FROM skill_offerings o
    JOIN skills s ON o.skill_id = s.skill_id
    JOIN users u ON o.user_id = u.user_id
    JOIN user_profiles p ON u.user_id = p.user_id
    WHERE
        o.is_active = TRUE
        AND u.is_active = TRUE
        AND o.user_id != req_user_id  -- Can't match with your own offerings
        AND (
            o.skill_id = req_skill_id  -- Exact skill match
            OR
            s.category_id = (SELECT category_id FROM skills WHERE skill_id =
req_skill_id)  -- Same category
        )
    ORDER BY match_score DESC, o.created_at DESC;
END;
$$ LANGUAGE plpgsql;

-- 3. Procedure to create exchange from request and offering
CREATE OR REPLACE PROCEDURE create_exchange_from_request_offering(
    request_id_param INTEGER,
    offering_id_param INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    req_user_id INTEGER;
    offer_user_id INTEGER;
BEGIN
    -- Get the requester's user ID
    SELECT user_id INTO req_user_id
    FROM skill_requests
    WHERE request_id = request_id_param;
```

```
    -- Get the provider's user ID
    SELECT user_id INTO offer_user_id
    FROM skill_offerings
    WHERE offering_id = offering_id_param;

    -- Make sure they're different users
    IF req_user_id = offer_user_id THEN
        RAISE EXCEPTION 'A user cannot exchange with themselves';
    END IF;

    -- Insert the exchange
    INSERT INTO exchanges (
        provider_id,
        requester_id,
        offering_id,
        request_id,
        status
    )
    VALUES (
        offer_user_id,
        req_user_id,
        offering_id_param,
        request_id_param,
        'pending'
    );

    -- Update the request to mark it as being addressed
    UPDATE skill_requests
    SET is_active = FALSE, updated_at = CURRENT_TIMESTAMP
    WHERE request_id = request_id_param;
END;
$$;

-- 4. Trigger to update timestamps
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_users_timestamp
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();
```

```sql
CREATE TRIGGER update_user_profiles_timestamp
BEFORE UPDATE ON user_profiles
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();

CREATE TRIGGER update_skill_offerings_timestamp
BEFORE UPDATE ON skill_offerings
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();

CREATE TRIGGER update_skill_requests_timestamp
BEFORE UPDATE ON skill_requests
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();

CREATE TRIGGER update_exchanges_timestamp
BEFORE UPDATE ON exchanges
FOR EACH ROW
EXECUTE FUNCTION update_timestamp();

-- 5. Trigger to validate user emails before insert/update
CREATE OR REPLACE FUNCTION validate_aui_email()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.email NOT LIKE '%@aui.ma' THEN
        RAISE EXCEPTION 'Email must be from the AUI domain (@aui.ma)';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_email_domain
BEFORE INSERT OR UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION validate_aui_email();

-- 6. Trigger to ensure provider and requester are different in exchanges
CREATE OR REPLACE FUNCTION validate_exchange_users()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.provider_id = NEW.requester_id THEN
        RAISE EXCEPTION 'Provider and requester must be different users';
    END IF;
    RETURN NEW;
END;
```

```
$$ LANGUAGE plpgsql;

CREATE TRIGGER check_exchange_users
BEFORE INSERT OR UPDATE ON exchanges
FOR EACH ROW
EXECUTE FUNCTION validate_exchange_users();

-- 7. Function to search skills by keywords
CREATE OR REPLACE FUNCTION search_skills(search_term TEXT)
RETURNS TABLE(
    skill_id INTEGER,
    skill_name VARCHAR,
    category_name VARCHAR,
    offering_count BIGINT,
    request_count BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        s.skill_id,
        s.name AS skill_name,
        c.name AS category_name,
        COUNT(DISTINCT o.offering_id) AS offering_count,
        COUNT(DISTINCT r.request_id) AS request_count
    FROM skills s
    JOIN categories c ON s.category_id = c.category_id
    LEFT JOIN skill_offerings o ON s.skill_id = o.skill_id AND o.is_active =
TRUE
    LEFT JOIN skill_requests r ON s.skill_id = r.skill_id AND r.is_active =
TRUE
    WHERE
        LOWER(s.name) LIKE '%' || LOWER(search_term) || '%' OR
        LOWER(s.description) LIKE '%' || LOWER(search_term) || '%' OR
        LOWER(c.name) LIKE '%' || LOWER(search_term) || '%'
    GROUP BY s.skill_id, s.name, c.name
    ORDER BY
        CASE
            WHEN LOWER(s.name) LIKE LOWER(search_term) || '%' THEN 1
            WHEN LOWER(s.name) LIKE '%' || LOWER(search_term) || '%' THEN 2
            ELSE 3
        END,
        (COUNT(DISTINCT o.offering_id) + COUNT(DISTINCT r.request_id)) DESC;
END;
$$ LANGUAGE plpgsql;
```

This comprehensive set of PostgreSQL queries covers all aspects of the SkillSphere database implementation, from creation to data manipulation, views, and procedural logic. The design follows best practices for PostgreSQL and implements all the requirements specified in the project proposal.